Sparse Tensor-Based Multiscale Representation for Point Cloud Geometry Compression

Jianqiang Wang, Dandan Ding, Zhu Li, *Senior Member, IEEE*, Xiaoxing Feng, Chuntong Cao, and Zhan Ma, *Senior Member, IEEE*

Abstract—This study develops a unified Point Cloud Geometry (PCG) compression method through the processing of multiscale sparse tensor-based voxelized PCG. We call this compression method SparsePCGC. The proposed SparsePCGC is a low complexity solution because it only performs the convolutions on sparsely-distributed Most-Probable Positively-Occupied Voxels (MP-POV). The multiscale representation also allows us to compress scale-wise MP-POVs by exploiting cross-scale and same-scale correlations extensively and flexibly. The overall compression efficiency highly depends on the accuracy of estimated occupancy probability for each MP-POV. Thus, we first design the Sparse Convolution-based Neural Network (SparseCNN) which stacks sparse convolutions and voxel sampling to best characterize and embed spatial correlations. We then develop the SparseCNN-based Occupancy Probability Approximation (SOPA) model to estimate the occupancy probability either in a single-stage manner only using the cross-scale correlation, or in a multi-stage manner by exploiting stage-wise correlation among same-scale neighbors. Besides, we also suggest the SparseCNN based Local Neighborhood Embedding (SLNE) to aggregate local variations as spatial priors in feature attribute to improve the SOPA. Our unified approach not only shows state-of-the-art performance in both lossless and lossy compression modes across a variety of datasets including the dense object PCGs (8iVFB, Owlii, MUVB) and sparse LiDAR PCGs (KITTI, Ford) when compared with standardized MPEG G-PCC and other prevalent learning-based schemes, but also has low complexity which is attractive to practical applications.

Index Terms—Point cloud geometry compression, sparse tensor, sparse convolution, multiscale representation, occupancy probability approximation, neighborhood embedding

1 Introduction

Due to their outstanding flexibility for representing 3D objects realistically and naturally, point clouds have become a popular media format used in a large number of applications, such as the Augmented/Virtual Reality (AR/VR), autonomous driving, and cultural e-heritage. This then raises an urgent need for high-efficiency lossless and lossy compression of point clouds [1]. This work first exemplifies the development of *lossless compression* in detail, since existing industrial applications demand such functionality for archiving point cloud-based digital assets efficiently, such as the 3D building information models used in digital twin [2], and for enforcing safety-critical system using high-precision 3D point cloud maps in autonomous robots. Later then, we show that the same architecture can be easily extended for *lossy compression* as well.

A point cloud is a collection of non-uniformly and sparsely distributed points that can be characterized using their 3D coordinates (e.g., (x, y, z)) and attributes (e.g., RGB colors, reflectances) if applicable. Unlike those well-structured pixel grids of a 2D image plane or a video frame, a point cloud relies on unconstrained displacement of points to represent arbitrary-shaped 3D objects flexibly. However, this creates issues for the efficient coding of geometric occupancy due to difficulties in characterizing and exploiting inter-correlation across irregularly scattered points in a free 3D space. The main focus of this paper is Point Cloud Geometry (PCG) compression. For the sake of simplicity, we start our discussion from the lossless mode of dense object Point Cloud Geometry Compression (PCGC), and then extend the studies to other scenarios.

1.1 Motivation

Usually, the compression efficiency of a point in a point cloud is closely related to its probability approximation conditioned on available neighbors [3], [4]. The entropy $R_{\vec{v}_k}$ after compression can be represented as:

$$R_{\vec{v}_k} = \mathbb{E}\big[-\log_2 p_{\vec{v}}(\vec{v}_k|\vec{v}_{k-1}, \vec{v}_{k-2} \dots \vec{v}_{k_0})\big]. \tag{1}$$

Here, $\vec{v}_k$ is the kth to-be-compressed point, and $\vec{v}_{k-1}$, $\vec{v}_{k-2}$, $\dots \vec{v}_{k_0}$ are its causal neighbors that are already processed and served as the prior knowledge for the process of current

- *Jianqiang Wang and Zhan Ma are with the School of Electronic Science and Engineering, Nanjing University, Nanjing, Jiangsu 210093, China. E-mail: wangjq@smail.nju.edu.cn, mazhan@nju.edu.cn.*
- *Dandan Ding is with the HangZhou Normal University, Hangzhou, Zhejiang 310030, China. E-mail: dandanding@hznu.edu.cn.*
- *Zhu Li is with the University of Missouri, Kansas City, MO 64110 USA. E-mail: zhu.li@ieee.org.*
- *Xiaoxing Feng and Chuntong Cao are with the Jiangsu Longyuan Zhenhua Marine Engineering Co., LTD., Nantong, Jiangsu 226000, China. E-mail: fengxiaoxing@zpmc.com, 25519785@qq.com.*

Manuscript received 18 November 2021; revised 26 September 2022; accepted 24 November 2022. Date of publication 1 December 2022; date of current version 5 June 2023.
This work was supported by the National Natural Science Foundation of China under Grants 62022038 and U20A20184.
(Corresponding author: Zhan Ma.)
Recommended for acceptance by P. Mordohai.
Digital Object Identifier no. 10.1109/TPAMI.2022.3225816

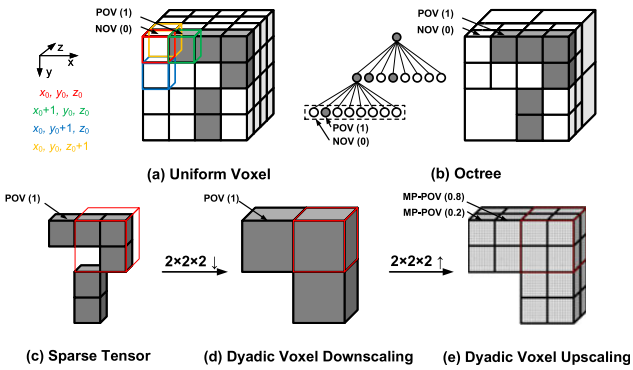


Fig. 1. *Various Representation Models of Voxelized PCG:* (a) uniform voxel; (b) octree model (and parent-tree layout); (c) sparse tensor; (d) dyadic voxel downscaling $2 \times 2 \times 2 \downarrow$ from (c); (e) dyadic voxel upscaling $2 \times 2 \times 2 \uparrow$ from (d) to generate MP-POVs and associated occupancy probability, i.e., MP-POV($p_{\text{MP-POV}}$). Voxel grids with the solid line are all involved in computation. *Solid grey voxels are positively occupied, a.k.a., POVs; and the rest are NOVs, in (a) and (b). The red box in (c), (d), and (e) contains voxels used in resampling. Voxels painted with dense grid pattern are MP-POVs.*

kth element. $p_{\vec{v}}(\vec{v}_k|\vec{v}_{k-1}, \vec{v}_{k-2} \ldots \vec{v}_{k_0})$ is the conditional probability of $\vec{v}_k$ which is determined by a context model and used to approximate $R_{\vec{v}_k}$. The more precise the context modeling is, the closer the probability distribution to the real data is, and the fewer bits are consumed.

To build adaptive contexts using spatial neighbors, a number of 3D representation models have been developed, including the "uniform voxel model" in Fig. 1a and "octree model" in Fig. 1b [5], on top of which either rules-based [1], [6] or deep neural network (DNN)-based context models [7], [8], [9], [10], [11], [12], [13] can be devised to explore correlations across spatial neighbors.

Although those DNN models improve the compression efficiency noticeably against rules-based approaches like recently-approved ISO/IEC MPEG (Moving Picture Experts Group) Geometry-based PCC (G-PCC) standard [1], [6], their huge complexity hinders the prompt use of these models in practice. On the other hand, alternative point-wise processing methods[1] [14], [15], [16] like PointNet++ in [16] report low complexity, but their compression performance largely suffers [15]. More importantly, these learning-based approaches cannot easily generalize themselves to various compression scenarios, such as the support of both dense and sparse point clouds and the support of both lossy and lossless modes.

After the conclusion of G-PCC [6], international standardization groups, such as the ISO/IEC MPEG, and ISO/IEC JPEG (Joint Photographic Experts Group) are actively pursuing the next-generation point cloud compression using learning-based approaches [17], [18], which urges a solution with *better coding efficiency*, *affordable complexity*, and *robust* model generalization at the same time. Towards this goal, a more efficient 3D representation method and a better spatial neighbor utilization mechanism for adaptive contexts are highly desired.

1.2 Our Approach

Following the conventions used in standardized G-PCC [1] and other prevalent learning-based PCGC methods [7], [8], [9], this study uses voxelized PCG for processing. A volumetric presentation of a PCG is depicted in Fig. 1a using a densely-sampled uniform voxel grid which is referred to as the "uniform voxel" representation for convenience. We then use *binary occupancy status* (or occupancy status) to describe whether the geometric position of the current voxel is positively occupied or not. For those Positively-Occupied Voxels (POVs) painted in solid grey in Fig. 1, they are marked using indicators "1", e.g., POV(1); Conversely, Non-Occupied (empty) Voxels (NOVs) in white box are indicated using "0", e.g., NOV(0). Apparently, those POVs are converted from raw "points" of an original PCG by voxelization.

1.2.1 Multiscale Sparse Tensor Representation of PCG

Facing those non-uniformly distributed POVs of an input PCG, this paper suggests the use of *Sparse Tensor* to best exploit the sparsity of POVs [19]. Through the use of sparse tensor, we only cache the geometric coordinates and attributes (if applicable) for Most-Probable POVs, i.e., MP-POVs. These MP-POVs are generated from POVs of preceding lower scale via voxel upscaling, as illustrated from Fig. 1d to 1e, to form scale-wise sparse tensor under a *multiscale representation* framework.

With this aim, we progressively downscale the original PCG into multi-resolution PCGs and upscale them correspondingly for hierarchical reconstruction, as in Fig. 2. We simply use *dyadic voxel sampling* to upscale or downscale related sparse tensor, which basically performs the scaling with a stride of 2 at each axis (i.e., x-, y- and z-axis in a Cartesian coordinate system). We regard the $2 \times 2 \times 2 \uparrow$ (or $2^3 \uparrow$) as dyadic voxel upscaling, and $2 \times 2 \times 2 \downarrow$ (or $2^3 \downarrow$) as dyadic voxel downscaling.

- As for the downscaling in encoding, we merge eight connected sub-voxels in a $2 \times 2 \times 2$ voxelized box to a single voxel (from Fig. 1c to 1d). If any one out of these eight sub-voxels is a POV, the merged voxel is a POV. In the encoder, we have full knowledge of each POV at each scale.
- As for the upscaling in decoding, we sub-divide a specific voxel to eight sub-voxels (from Fig. 1d to 1e). If this voxel is a POV[2], its 8 children are marked as MP-POVs since we cannot decide which sub-voxel is positively occupied just right after the upscaling. The collection of MP-POVs is a superset of POVs.

1.2.2 Cross-Scale Context Modeling

The proposed SparsePCGC encodes the occupancy of MP-POVs at each scale into a compressed bitstream and correspondingly it decodes the binary bits to reconstruct same-scale POVs. For either lossy or lossless compression, the

1. Note that aforementioned uniform voxel representation and octree decomposition do require the voxelization to pre-process raw input, while currently these point-wise methods do not necessarily need this step.

2. We know the ground truth after decoding in lossless compression mode because the sparse tensor in encoder and the decoded tensor in decoder have to be the same.

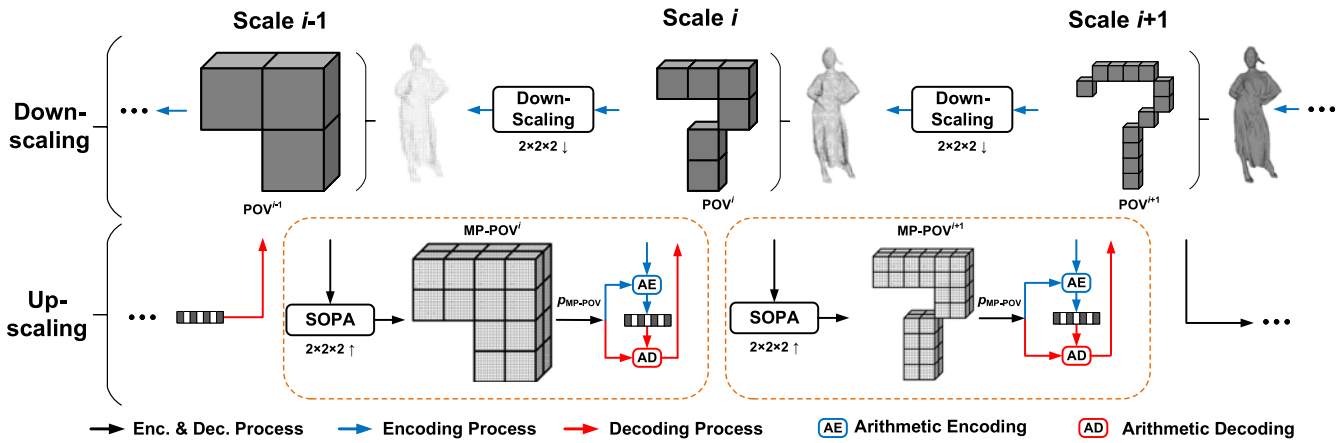


Fig. 2. *SparsePCGC.* In the encoding process (blue arrows), we progressively downscale the PCG tensor and encode occupied voxels at each scale into the binary stream according to their occupancy probability; As for the decoding flow (red arrows), we exactly reconstruct occupied voxels, e.g., POV or NOV, by decoding the binary bits with the occupancy probability. The key component, a.k.a., occupancy probability approximation, that plays a vital role for compression efficiency is developed by utilizing the cross-scale and same-scale priors and is plugged in both encoder and decoder for bit-wise match. Such SparseCNN-based Occupancy Probability Approximation is referred to as the "SOPA". The SOPA can be fulfilled through either single-stage or multi-stage manner, depending on the tradeoff between the performance and complexity in applications.

occupancy probability approximation of each MP-POV is critical to the performance of SparsePCGC.

Upon the multiscale representation model, we perform the cross-scale context modeling by utilizing decoded POVs (e.g., occupancy status and feature attribute if applicable) from the preceding lower scale to approximate the occupancy probability of each MP-POV at the current scale. Note that the cross-scale context modeling is limited between two consecutive scales only. This allows us to train models just using samples from two consecutive scales and then apply them for inference across all scales. This cross-scale model sharing strategy reduces the model size significantly.

The proposed cross-scale context modeling is fulfilled by SparseCNN-based Occupancy Probability Approximation (SOPA) that can be implemented in various ways. For example, we can approximate the occupancy probabilities of eight upscaled MP-POVs in one shot through the use of a single-stage SOPA. We can also exploit neighborhood correlation by applying multi-stage SOPA to progressively approximate the occupancy probabilities using previously-decoded POVs at the same scale from one group to another.

The aforementioned studies only explore correlations in the course of scale upsampling by applying the dyadic voxel downscaling in encoding without any spatial information embedding. Here we propose the SLNE (SparseCNN-based Local Neighborhood Embedding) to characterize and embed local variations of each POV as its feature attribute in downscaling. Thereafter, when performing the scale upsampling, both occupancy status and latent features of each decoded POV are fed into SOPA model for better performance.

1.3 Contribution

Our contributions are summarized as follows:

- Together with the preliminary exploration in [22], we are probably the *first one* to suggest the convolutional representation of multiscale sparse tensor for point cloud geometry compression, with which we effectively exploit correlations through cross-scale context modeling for better efficiency.
- The proposed SparsePCGC generalizes very well with state-of-the-art efficiency in both lossless and lossy scenarios across a variety of datasets, such as the densely-sampled 8i Voxelized Full Bodies (8iVFB) [23], Owlii sequences [24] and Microsoft Voxelized Upper Bodies (MVUB) [25], and sparsely-sampled LiDAR sequences KITTI and Ford, in comparison to the standardized G-PCC and other learning-based approaches [8], [9], [10], [11], [13], [22], [26], [27], [28].
- Our method also demonstrates low space and time complexity. For example, its runtime for encoding and decoding is quantitatively comparable to the G-PCC, requiring just 1∼2 seconds on average to encode/decode a large-scale PCG frame (see Table 2), which is about orders of magnitude reduction compared to recently-published VoxelDNN [29], NNOC [27], and OctAttention [28] (in decoding). Moreover, sharing the same model across scales reduces the space complexity significantly, which is attractive to industrial practitioners.

Table 1 lists notations and abbreviations frequently used in this work for convenience.

2 Related Work

This section reviews relevant techniques for the compression of PCG.

2.1 Rules-Based Approaches

The octree model is the most popular format to represent voxelized point cloud [30]. A volumetric point cloud is recursively divided using octree decomposition until it reaches the leaf nodes/voxels. The occupancy of a voxel or sub-voxel can be then statistically compressed through predefined context prediction following the parent-child tree structure (see Fig. 1b) [31], [32]. This octree-based coding mechanism using handcrafted rules was adopted in MPEG G-PCC [1], known as the *octree geometry codec*.

TABLE 1
Notations and Abbreviations

Abbreviation	Description
PCC	Point Cloud Compression
PCG	Point Cloud Geometry
PCGC	Point Cloud Geometry Compression
POV	Positively-Occupied Voxel
MP-POV	Most Probable POV
NOV	Non-Occupied Voxel
SparseCNN	Sparse Convolution-based Neural Networks
SOPA	SparseCNN-based Occupancy Probability Approximation
SOPA (Position)	SparseCNN-based Occupancy Position Adjustment
SLNE	SparseCNN-based Local Neighborhood Embedding
G-PCC	Geometry-based PCC [20]
BD-Rate	Bjøntegaard Delta Rate [21]

To reconstruct object surfaces with finer structural details, a number of triangle meshes can be constructed locally on top of the octree model [33]. MPEG G-PCC then included this triangulation-based mesh model, a.k.a., triangle soups representation of local geometry, into the test model, referred to as the *trisoup geometry codec*.

Alternatively, traditional 2D image and video coding techniques [34] have demonstrated superior compression efficiency and have been used in many applications. This motivates us to project a given 3D object to multiple 2D planes from a variety of viewpoints and then leverage popular image and video codecs for compact representation of projected 2D image sequences. Examples include the MPEG Video-based PCC [33], View-PCC [35], etc. Such 3D-to-2D projection based solutions explore a different route which is not the focus of this work.

2.2 Learning-Based Methods

Numerous works have demonstrated that DNNs can best exploit inter-voxel correlations upon a specific 3D representation format for PCGC in either lossy or lossless mode.

Uniform Voxel Representation. Earlier attempts, including Wang et al. [8], Quach et al. [7], and Guarda et al. [9] apply the uniform voxel model in Fig. 1a to represent voxelized PCG, in which 3D dense convolutions can be directly applied to capture spatial correlations across voxels within the receptive field. These 3D convolutional layers are often stacked in an autoencoder architecture with quantized latent features at the bottleneck layer compressed using an adaptive arithmetic coder. For any given feature element at the bottleneck, its hyperprior and autoregressive neighbors can be utilized for probability estimation [8].

Octree Representation. As in the uniform voxel representation, empty voxels, i.e., NOVs, are treated equally as those POVs in computation, leading to unbearable consumption of storage and computation. Then, the octree model is revisited since it represents POVs from coarse to fine scales in a progressive manner. Huang et al. [11] and Biswas et al. [12] adopt the Multi-Layer Perceptron (MLP) to exploit the dependency between parent and child nodes for occupancy probability prediction.

Although the octree model utilizes the occupancy to decompose the input PCG adaptively, its efficiency is largely constrained because of noticeable bits overhead caused by signaling the deep tree hierarchy incurred by isolated sparse points.

Hybrid Model. In addition to solely relying on the parent-child dependency of the octree model, available sibling nodes can be organized under a uniform voxel representation to use dense 3D CNNs (Convolutional Neural Networks), or under a series of sequential nodes to apply the Transformer, for context modeling. Such hybrid model is extensively studied in [13], [28] with noticeable compression performance improvement.

Point-Wise Representation. Point-wise solutions directly compress raw points of input point cloud without voxelization. They typically apply the PointNet++ [16] or other MLP-based autoencoders. As reported by Gao et al. [15], point-wise approaches not only perform poorly at high bit rates, but also have difficulties to generalize themselves to large-scale point clouds, although they have a low complexity requirement.

As will be shown later, the proposed SparsePCGC can effectively solve problems in existing solutions with better coding efficiency, lower complexity, and robust generalization. To help the audience understand the proposed framework quickly, we will review the processing of sparse tensor next.

2.3 Sparse Tensor Processing

2.3.1 Sparse Tensor

The number of POVs or MP-POVs is just a small percentage of total voxels in a volumetric PCG, exhibiting very sparse distribution in a 3D space. Thus, we choose to use the *Sparse Tensor* in Fig. 1c to represent a voxelized PCG, with which a hash map is used to index the geometric coordinates (and associated attributes if applicable) of those MP-POVs and an auxiliary data structure is applied to manage their connections efficiently [36]. As such, we greatly reduce the time complexity by just performing the computations on sparse MP-POVs to aggregate and embed information from available spatial neighbors.

By contrast, the octree model is limited by its parent-child hierarchy, which is difficult to represent spatial neighbors unless we explicitly construct such local spatial connections through the use of CNN or Transformer as in [13], [28]; while the uniform voxel model processes all voxels regardless of their occupancy status, which is redundant obviously.

2.3.2 Sparse Convolution

3D sparse convolution can be applied on sparse tensors efficiently. It is similar to commonly-used 3D dense convolution but only convolves using valid MP-POVs, which makes full use of the sparse characteristics of point clouds.

There are a variety of implementations of sparse convolution, e.g., Sparse 3D convolution [19], Sparse Blocks Network [37], SpConv [38], Submanifold Sparse ConvNet (Convolutional Network) [39], and 4D Spatiotemporal ConvNets (Minkowski CNN) [36]. This work chooses the 4D Spatiotemporal ConvNets compliant MinkowskiEngine [36] to demonstrate the SparsePCGC due to its wide support of

different operators and platforms. Other implementations can be used as well, for instance, recently-released TorchSparse[40] that offers 1.6× speedup of the MinkowskiEngine [36] can be a promising option for much faster prototype.

A sparse tensor can be formulated using a set of coordinates $\vec{C} = \{(x_i, y_i, z_i)\}_i$ and associated features $\vec{\mathbf{F}} = \{\vec{f}_i\}_i$. Thus the sparse convolution is formulated as :

$$\vec{f}_u^{out} = \sum\nolimits_{k \in \mathbb{N}^3(u, \vec{C}^{in})} W_k \vec{f}_{u+k}^{in} \quad \text{for} \quad u \in \vec{C}^{out}, \tag{2}$$

where $\vec{C}^{in}$ and $\vec{C}^{out}$ are input and output coordinates. $\vec{C}^{in}$ and $\vec{C}^{out}$ are the same if the resolution is kept. $\vec{f}_u^{in}$ and $\vec{f}_u^{out}$ are input and output feature vectors at coordinate u (a.k.a., (x_u, y_u, z_u)), respectively. $\mathbb{N}^3(u, \vec{C}^{in}) = \{k | u + k \in \vec{C}^{in}, k \in \mathbb{N}^3\}$ defines a 3D convolutional kernel centered at u with offset k in $\vec{C}^{in}$. By setting different k, we can easily retrieve neighboring POVs or MP-POVs within a predefined 3D receptive field, e.g., $k \times k \times k$ or k^3, for information aggregation. k is set to 3 or 5 for lightweight computation. W_i is kernel weights.

Clearly, both space and time complexity is greatly reduced when using sparse convolution instead of regular dense convolutions. More details about the implementation of sparse convolution used in this work can be found in [36], [41]. Just as a quantitative reference, our GPU-based SparsePCGC requires less encoding and decoding runtime than the CPU-based G-PCC anchor, consuming just a few percentage of those uniform voxel based methods [10], [29] (see Table 4). In the meantime, our method requires a small amount of storage space due to the model sharing strategy across different scales (see Sec. 6.4).

2.3.3 Notation

Sparse convolution (SConv) or transposed sparse convolution (TSConv) can be formatted using "K, C, S", where $K = k \times k \times k$ (k^3) is the receptive field of 3D convolution, C describes the number of channels, and S can be $s \times s \times s \downarrow$ ($s^3 \downarrow$) for downscaling or $s \times s \times s \uparrow$ ($s^3 \uparrow$) for upscaling. Note that downscaling (upscaling) operator is associated with the SConv (TSConv) accordingly. Except for voxel sampling layer in Fig. 3a, resolution scaling is not required for most cases with $s = 1$, we can then simplify the notation to "SConv k^3, C" by omitting the S.

3 Multiscale Sparse Tensor-Based PCGC

This section first pictures the overall system design and then offers technical details of the proposed SparsePCGC.

3.1 Framework

A general framework of multiscale sparse tensor-based PCGC is illustrated in Fig. 2. We call it SparsePCGC for short[3]. It includes a pair of encoder and decoder, where the encoder inputs the PCG to generate the compressed bitstream, and the decoder parses the bitstream to reconstruct the original PCG.

3. Note that we first exemplify the idea using lossless PCGC; then, the proposed architecture can be easily extended to support lossy PCGC. Our preliminary results on lossy PCGC can also be found in [22].

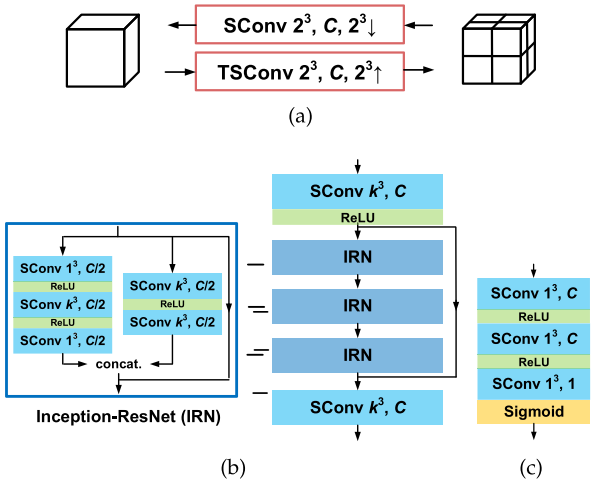


Fig. 3. *Illustration of Basic SparseCNN Modules.* (a) Voxel Sampling Layer (VSL) applied to upscale or downscale voxels dyadically through the use of "SConv $2^3, C, 2^3 \downarrow$" or "TSConv $2^3, C, 2^3 \uparrow$"; (b) Deep Feature Aggregation (DFA) used to characterize and embed information from spatial neighbors within the receptive field; (c) Occupancy Output Layer (OOL) for probability or offset derivation.

The framework contains a sequence of dyadic voxel sampling, e.g., $S = 2 \times 2 \times 2 \uparrow$ as the upscaling, or $S = 2 \times 2 \times 2 \downarrow$ as the downscaling. They can be integrated with the SparseCNN blocks for better information embedding during the resampling. Such a progressive scaling mechanism enables the multiscale representation so that we can support resolution scalability easily. Since we enforce the identical processing between two adjacent scales, the overall SparsePCGC can be comprehensively explained through a two-scale example from the (i-1)th scale to the ith scale as in Fig. 2.

As seen, each POV from the preceding scale will be subdivided into eight MP-POVs (e.g., expanding from Fig. 1d to 1e), of which some may be POVs and the rest are NOVs. The ground truth is known in the encoder but unknown in the decoder. At each scale, we need to compress the occupancy status of each MP-POV in the encoder, and reversely reconstruct POVs from upscaled MP-POVs in the decoder by interpreting compressed syntax. Thus, the same cross-scale context model is shared between encoder and decoder, which is facilitated by the SOPA using decoded POVs in the preceding scale to generate occupancy probability $p_{\text{MP-POV}}$ of the MP-POV in upscaled tensor for arithmetic coding.

Next, we first brief basic SparseCNN blocks used for SOPA computation in SparsePCGC.

3.2 Basic SparseCNN Blocks

Convolutional and (nonlinear) activation layers are generally stacked to form SparseCNN blocks, including voxel sampling layer (VSL), deep feature aggregation (DFA), and occupancy output layer (OOL) shown in Fig. 3. All computations are performed using sparsely distributed POVs or MP-POVs in a sparse tensor.

Voxel Sampling Layer (VSL). We embed voxel upscaling in SOPA and voxel downscaling in SLNE[4] to execute the cross-scale message passing. For the downscaling in SLNE

4. If we do not include the SLNE in the encoder, simple dyadic voxel downscaling is used.

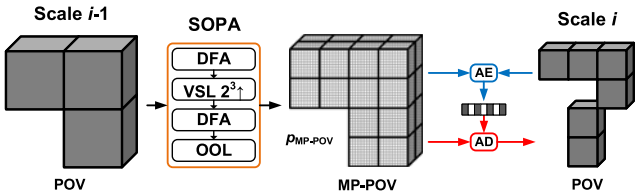


Fig. 4. *One-Stage SOPA*. Each decoded POV is processed by the SOPA engine to generate eight MP-POVs and associated $p_{\text{MP-POV}}$s. The encoder uses $p_{\text{MP-POV}}$s to encode ground truth into the bitstream while the decoder determines true POVs and NOVs by decoding the bitstream with $p_{\text{MP-POV}}$s.

illustrated in Fig. 7, we apply the "SConv 2^3, C, $2^3\downarrow$" to merge eight spatially-connected voxels (e.g., $2\times2\times2$) into a single voxel, as shown in Fig. 3a which is noted as "VSL $2^3\downarrow$" for short. For the upscaling in SOPA depicted in Figs. 4 and 6, we apply the "TSConv 2^3, C, $2^3\uparrow$" to subdivide a voxel into eight sub-voxels (or child nodes), i.e., VSL $2^3\uparrow$. C is an adjustable model parameter standing for the number of channels.

Deep Feature Aggregation (DFA). For most convolutional layers without resolution scaling, we typically apply the "SConv k^3, C" to aggregate features of neighboring voxels at the same scale. We set k to 3 for dense object point clouds and 5 for sparse LiDAR point clouds because a slightly larger convolutional kernel is more beneficial for much sparser LiDAR PCGs to include sufficient neighbors for information embedding. Increasing k can improve the compression efficiency at the increase of both space and time complexity. The number of channels (C) is set to 32. We use a deep Inception ResNet (IRN) block that is comprised of three basic IRN units [42] to characterize the spatial dependency across voxel neighbors. The IRN is also used in our earlier works, demonstrating convincing efficiency and affordable complexity [8], [22].

Occupancy Output Layer (OOL). As for SOPA models detailed subsequently, the last occupancy output layer (OOL) is applied, which generally consists of three convolutional layers and a Sigmoid activation layer to derive the occupancy probability p in the range of [0,1] for dense object PCG, as illustrated in Fig. 3c. On the other hand, for the compression of sparse LiDAR point clouds, the OOL embedded in SOPA (Position) model removes the Sigmoid layer and expands the number of output channels of the last convolutional layer from 1 in Fig. 3c to 3 in Fig. 8b to derive three coordinate offsets directly.

As will be unfolded later, the aforementioned VSL, DFA, and OOL blocks will be deliberately stacked to form the SOPA, SOPA (Position), and SLNE modules used in SparsePCGC.

3.3 Lossless SOPA: Exploiting Spatial Priors For More Compact Representation

The compression performance of SparsePCGC is mainly determined by the efficiency of underlying SOPA engine for context modeling. Even for SLNE, it is also utilized to improve the SOPA for a more accurate probability approximation. We start our discussion from a toy baseline.

3.3.1 A Toy Baseline Using One-Stage SOPA Only

This example only allows dyadic voxel downscaling (i.e., no SLNEs) in encoder, and ultimately relies on the One-Stage SOPA for MP-POV probability (i.e., $p_{\text{MP-POV}}$) estimation. As shown in Fig. 4, it inputs each POV from preceding scale to generate $p_{\text{MP-POV}}$s of corresponding upscaled 8 MP-POVs. Such one-stage SOPA applies the "TSConv 2^3, C, $2^3\uparrow$" in VSL, i.e., VSL $2^3\uparrow$, for resolution upscaling. In fact, we cannot learn sufficient information if we use the geometry information of preceding lower-scale POVs only for $p_{\text{MP-POV}}$ estimation in such a one-shot manner, yielding inferior coding efficiency to the G-PCC anchor (i.e., average 5.5% compression ratio increase in Table 2 for lossless coding).

3.3.2 Multi-Stage SOPA

Previous One-Stage SOPA assumes that eight MP-POVs upscaled from a POV of the last scale are independent of each other. However, these MP-POVs are closely correlated because they are generated from the same POV and are present in close proximity. Hence, this section develops the Multi-Stage SOPA to progressively estimate the $p_{\text{MP-POV}}$s by exploiting the correlations of preceding lower-scale POVs and same-scale causal neighbors. The causal neighbors are already processed and serve as the prior knowledge for the process of current elements. They are also widely used in learning-based image/video coding [43], [44] for more accurate probability estimation.

Grouping Arrangement for Multi-Stage Processing. Hereafter, the Multi-Stage SOPA is mainly about how to arrange MP-POV groups (e.g., elements from G1 to G8 in Fig. 5b) for inter-group dependency exploration. We have exemplified three typical grouping arrangements in Fig. 5b, e.g., a One-Stage example by processing all eight groups in parallel at

TABLE 2
Compression Performance Measured by Bits per Point (bpp) and Complexity (Runtime in Seconds) Tradeoff for Various SOPA Models in Lossless Mode Using 8iVFB Samples

PCGs	G-PCC	One-stage	3-Stage	8-Stage	n-Stage	SLNE Enh. One-Stage	SLNE Enh. 3-Stage
Longdress	1.015	1.015	0.69	0.623	0.608	0.752	0.677
Loot	0.970	1.032	0.66	0.594	0.581	0.713	0.546
Red&balck	1.010	1.156	0.758	0.688	0.69	0.854	0.756
Soldier	1.030	1.103	0.702	0.627	0.62	0.764	0.684
Average	1.029	1.086	0.703	0.633	**0.625**	0.77	0.666
Gain	-	+5.5%	-31.7%	-38.5%	**-39.3%**	-25.1%	-35.3%
EncTime (s)	5.88	**0.66**	1.09	1.86	0.72	1.17	1.54
DecTime (s)	3.34	**0.65**	1.00	1.78	2 hr	0.92	1.18

Encoding/decoding time is averaged for each frame. $k = 3$ & $C = 32$ in sparse convolution.

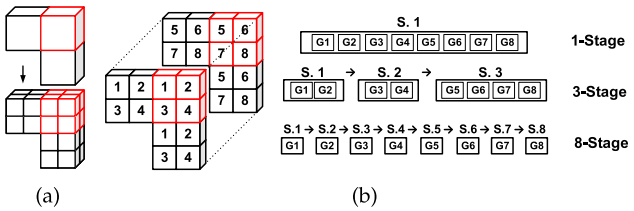


Fig. 5. *Grouping Arrangement for Multi-Stage Processing.* (a) dyadic resolution upscaling: one POV is upsampled to eight MP-POVs (or child nodes as illustrated by octree expansion); (b) 1/3/8-Stage examples by grouping eight labeled MP-POVs accordingly. Numbers with symbol G correspond to group labels, i.e., elements marked with "i" belong to the ith group, e.g., Gi, i =1, 2, ..., 8. Elements in different groups can be categorized for parallel processing at each stage (e.g., S.j).

the same stage "S. 1" and an 8-Stage example to process element groups from "S. 1" to "S. 8" stage by stage. Different groups can be arranged to fulfill the multistage computing, and element groups at the same stage are processed concurrently. To best exploit neighborhood correlation at the same scale, (grouped) POVs that have been just determined in preceding stages are utilized as priors to process MP-POVs in succeeding stages. Next, we use the 8-Stage example (a.k.a., eight groups) to explain the proposed Multi-Stage SOPA. The same methodology can be extended to other grouping schemes easily.

In the companion supplemental material, which can be found on the Computer Society Digital Library at http://doi.ieeecomputersociety.org/10.1109/TPAMI.2022.3225816, we explain the grouping arrangement for n-Stage SOPA, where elements of all upscaled MP-POVs are processed sequentially in a raster scan order. The n-Stage SOPA fully relies on the autoregressive context model for probability estimation, imposing strict causal data dependency in decoding, which yields unbearable runtime (see Table 2).

8-Stage SOPA. Given any "8 MP-POVs" set upsampled from the corresponding POV of the preceding scale, an intuitive scheme is to categorize each element into an individual group, e.g., G1, G2, ..., G8 as shown in Fig. 5b. The same labeling is applied to all "8 MP-POVs" sets. Therefore, MP-POVs labeled with "1", i.e., G1 MP-POVs, will be processed simultaneously at the first stage (i.e., "S. 1"), and then MP-POVs labeled with "2" (a.k.a., G2 MP-POVs in the second stage "S. 2"), till we traverse all eight groups.

Fig. 6 details the 8-Stage SOPA step by step:

1) All MP-POVs are upscaled from corresponding POVs of preceding lower-scale through the use of stacked DFA and VSL blocks and are partitioned into eight groups from G1 to G8. Multi-Stage SOPA is applied to process grouped elements from the first group G1 to the last group G8, and elements in the same group are processed in parallel.
2) At the first stage, G1 MP-POVs are processed using stacked DFA and OOL blocks to determine their $p_{\text{MP-POV}}$s for compressing ground truth voxel occupancy information into the bitstream in encoder or parsing bitstream in decoder to identify POVs and NOVs. Note that decoded POVs and associated features (i.e., dark-grey painted "1" in Stage 2 of Fig. 6) are kept to process MP-POVs in succeeding stages, and the NOVs are pruned immediately, i.e., the upper-left "1" in Stage 1 is a NOV which is removed in Stage 2 of Fig. 6.
3) For the second and the remaining stages, computations are the same as "Stage 1" but with slightly different input data. As aforementioned, POVs and their features from preceding stages are used as priors to process MP-POVs in succeeding stages for better occupancy probability estimation.

For either Multi-Stage or One-Stage SOPA, we can approximate the bit rate of MP-POV occupancy status at the ith scale using

$$R_{\text{s}}(i) = \sum\nolimits_j -\log_2(p^i_{\text{MP-POV}}(j)), \tag{3}$$

where $p^i_{\text{MP-POV}}(j)$ is occupancy probability of the jth MP-POV at the ith scale derived using priors from preceding scale or stages.

By far, the SOPA model only uses the occupancy status (i.e., 1 or 0) of POVs from preceding scale to compute occupancy probability of current-scale MP-POVs through either a single-stage or a multi-stage manner. Next section will discuss possibilities to learn and embed spatial priors as the feature attribute in encoder to improve the SOPA efficiency.

3.3.3 SLNE Enhanced SOPA

Since we know ground truth labels at each scale in the encoder, we can learn and embed local spatial variations as priors in feature attribute. In this case, for each POV, besides its occupancy status signaling, these local variation features will be compressed and decoded to enhance the capacity of the SOPA engine. This section exemplifies the SLNE-enhanced One-Stage SOPA. Other combinations can be easily extended.

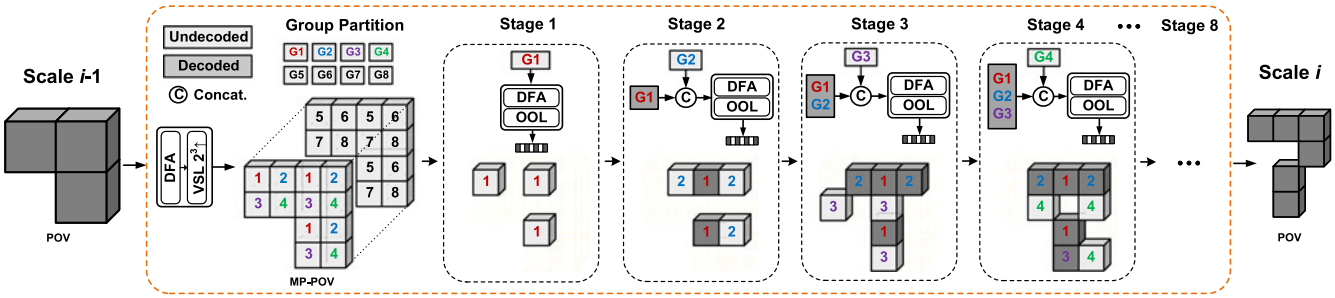


Fig. 6. *Multi-Stage SOPA.* 8-Stage SOPA is exemplified for cross-scale context modeling. At Stage n, MP-POVs in Gn are processed together with the POVs and their associated features that are determined in preceding stages, e.g., $n-1$, $n-2$, ..., for occupancy probability estimation. Note that the occupancy probability is used in both encoder and decoder.

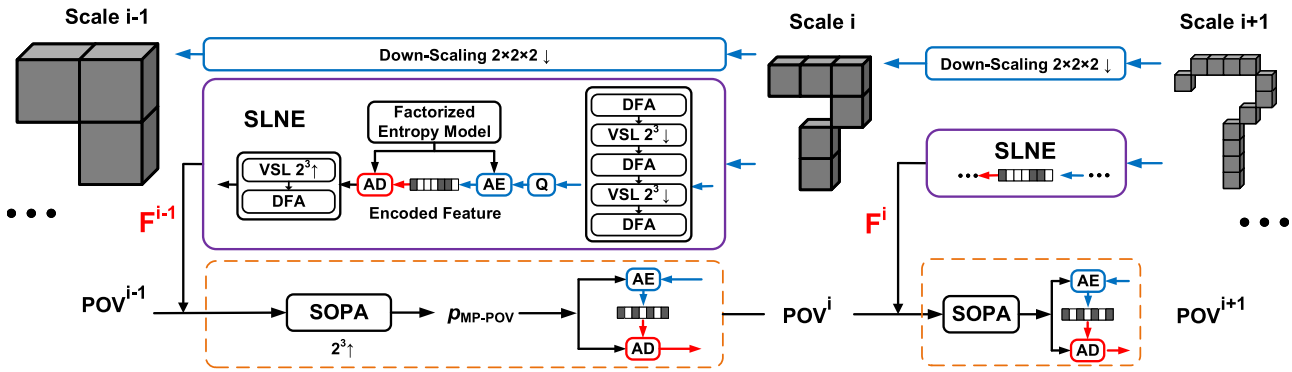


Fig. 7. *SLNE enhanced One-Stage SOPA.* A two-scale SLNE model interleaves three DFA and two "VSL $2^3\downarrow$" blocks for feature downscaling and embedding; correspondingly a pair of "VSL $2^3\uparrow$" and DFA block is used for feature upscaling. Quantization (Q) that is widely used in learning-based image/video coding [44], [45] is applied where the uniform noise injection is used in training and rounding is used in inference. The factorized entropy model is applied for the compression of features [45]. At each scale, the occupancy status and feature attribute of the sparse tensor are separately encoded and multiplexed into the bitstream.

Fig. 7 illustrates the SLNE between the ith and the $(i-1)$th scales. Similar to the SOPA model, the same SLNE is applied to any two adjacent scales. As seen, in addition to geometrically downscaling the sparse tensor $\mathbf{POV}^i$ to the $\mathbf{POV}^{i-1}$, the SLNE aggregates local neighborhood variations for each POV as its feature attributes $\mathbf{F}^{i-1}$. Thus, both occupancy status and feature attributes of each POV in $\mathbf{POV}^{i-1}$ are compressed into the bitstream, where the rate of occupancy status uses (3), and the rate of lossily coded feature attributes is

$$R_F(i-1) = \sum\nolimits_j -\log_2(p_{\mathbf{F}^{i-1}}(j)), \tag{4}$$

where $p_{\mathbf{F}^{i-1}}$ follows the factorized entropy model [45] with its parameters determined in training.

From the decoding point of view, both reconstructed occupancy status and features of each POV at the $(i-1)$th scale are fed into the SOPA engine to derive the occupancy probabilities of MP-POVs at the ith scale. The occupancy reconstruction is achieved by decoding the occupancy status related bitstream using estimated MP-POV probabilities; while the feature reconstruction stacks a DFA block and a VSL block to upscale decoded features by a factor of 2 in three axes accordingly.

As illustrated in Fig. 7, the SLNE related feature attribute processing, e.g., downscaling, encoding, decoding, and upscaling, are contained within two consecutive scales. Interleavedly stacking three DFA and two VSL blocks before quantization could well balance the compactness and efficiency of quantized features for optimally estimating the occupancy probability of MP-POVs in the ith scale tensor according to our extensive simulations. Having a pile of one DFA block and one VSL block to upscale decoded features is to match the geometry resolution of the sparse tensor of preceding lower scale, for instance, the $(i-1)$th scale in Fig. 7.

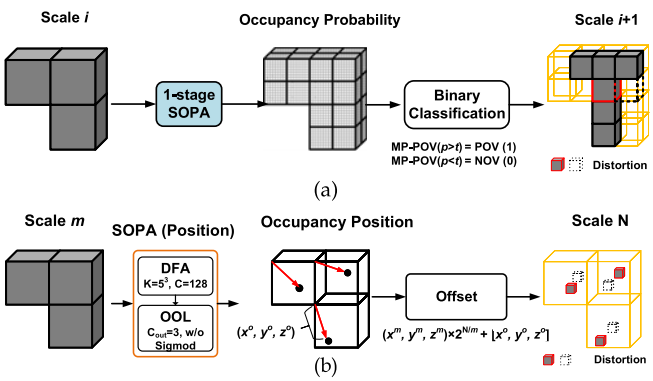


Fig. 8. *Lossy SOPA.* (a) Probability thresholding for dense object point clouds; (b) Position offset adjustment (POA) for sparse LiDAR point clouds.

4 Unified Lossless and Lossy Compression

Past sections detail the SOPA variants in lossless mode. Here, we show that it can be extended to support lossy compression under a unified framework. A slight modification is that we apply different strategies in lossy SOPA engine for dense and sparse point clouds as they present very diverse characteristics, including point density, bit precision, etc.

4.1 Lossy SOPA

4.1.1 Probability Thresholding for Dense Point Clouds

Recalling that we use the OOL block in SOPA shown in Fig. 4 to derive the occupancy probability of each MP-POV in lossless mode, the lossy mode then simply applies a probability threshold t to classify and determine the binary occupancy status. For instance, as illustrated in Fig. 8a, if $p_{\text{MP-POV}} > t$, the MP-POV is a POV; otherwise it is a NOV. t is set adaptively according to the number of POVs at each scale, which is the same as our previous works [8], [22]. Apparently, distortion is inevitable due to false classification.

As for the Multi-Stage SOPA model, the probability estimation in the succeeding stage relies on the outcome of preceding stages, implying that a false classification at earlier stages may accumulate errors stage by stage and lead to unexpected distortions. Thus, we choose to use One-Stage SOPA for lossy compression in this work.

4.1.2 Position Offset Adjustment for Sparse Point Clouds

The proposed probability thresholding-based lossy SOPA works well for the compression of dense point clouds.

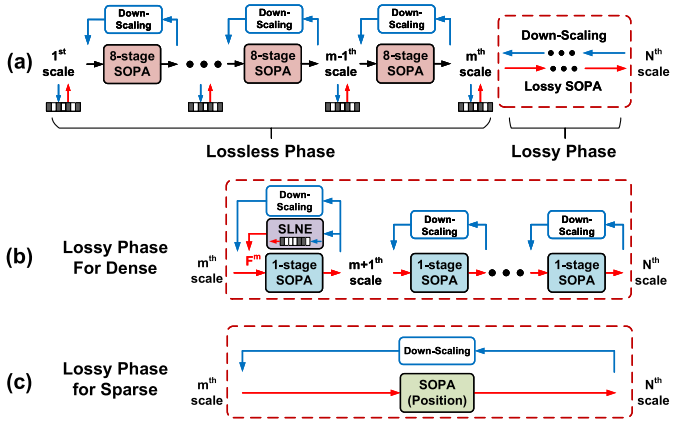


Fig. 9. *Lossy SparsePCGC.* (a) General architecture with m scales coded losslessly and $(N-m)$ scales coded in lossy mode; (b) lossy SOPA for dense object point clouds; (b) SOPA (Position) for sparse LiDAR point clouds;

However, it fails in LiDAR point clouds because of the extremely sparse distribution of points. Particularly at higher scales, occupancy estimation of the MP-POV is inefficient.

To solve this issue, we replace the occupancy *probability approximation* in the native SOPA model by occupancy *position adjustment*. For clarification, it is referred to as the "SOPA (Position)" in Fig. 8b. As seen, it consists of a DFA block and a modified OOL block to directly estimate the coordinate offsets. Compared with the SOPA model in Fig. 8a, the VSL block used for resolution scaling is removed. $k=5$ and $C=128$ are used to capture sufficient information from sparse LiDAR points. In the OOL block, the Sigmoid function is removed and the output layer is set with 3 channels to produce the position offsets for adjustment.

For a sparse tensor at the mth scale, we upscale it to the Nth scale in one-shot ($N > m$) as in Fig. 8b. For a given POV at (x^m, y^m, z^m) in the mth scale, its corresponding POV at the Nth scale locates at

$$(\hat{x}^N, \hat{y}^N, \hat{z}^N) = (x^m, y^m, z^m) \times 2^{\frac{N}{m}} + \lfloor x^o, y^o, z^o \rceil, \tag{5}$$

with (x^o, y^o, z^o) as the position offset estimated from the proposed SOPA (Position).

Although the position adjustment expands the resolution, it does not increase the number of POVs, which well reflects the distribution characteristics of LiDAR sequences. Similar position offset adjustment methodology is also used in [13], where it is called "Coordinates Refinement Module (CRM)". Note that either position offset adjustment or CRM belongs to the post-processing technique, which is optional for compression performance evaluation [28].

4.2 Unified SparsePCGC Framework

Previous discussions focus on the development of models using two adjacent scales in either lossy or lossless mode. In practice, the SparsePCGC stacks a sequence of two-scale models to process dense or sparse PCG input, in both lossy and lossless modes.

In lossless mode, the solution is fairly straightforward: we apply lossless SOPA models to compress both dense and sparse PCGs across all scales. The compression ratio against standardized G-PCC anchor is used to evaluate the performance quantitatively. Comparisons with other learning-based solutions are also provided.

As for lossy mode illustrated in Fig. 9, the SparsePCGC devises the lossless coding mode from the first scale at the lowest resolution to the mth scale, and the lossy mode to encode the remaining scales. Here we assume the N scale in total. We adapt m to best balance the rate and distortion for lossy compression. As reported in [22], applying lossy coded PCG for all scales would produce severe degradation of reconstruction quality due to scale-by-scale refinement, but could not bring noticeable rate reduction.

Note that the highest scale N is typically determined by the geometry precision of input point clouds. For example, $N=10$ or 11 for dense 8iVFB [23] and Owlii [24], and $N=18$ for KITTI and Ford sequences.

5 Training

This section details the datasets, loss functions, and strategies used to train the proposed SparsePCGC.

5.1 Datasets

Large-scale, popular, and publicly accessible ShapeNet [46] and KITTI [47] are used to generate respective training datasets for dense object and sparse LiDAR point clouds.

- *ShapeNet* [46] is one of the most popular CAD model dataset for 3D objects. Its subset ShapeNetCore contains 55 common object categories with about 51,300 unique 3D mesh models. We densely sample points on raw meshes to generate point clouds, and then randomly rotated and quantized them with 8-bit geometry precision at each dimension.
- *KITTI* (SemanticKITTI) [47] is a large-scale LiDAR dataset used for semantic scene understanding. It contains 22 sequences, a total of 43,552 scans (or frames) of outdoor scenes collected using the Velodyne HDL-64E LiDAR sensor. There are around 120k points on average per frame. These raw floating-point coordinates are quantized to millimeter scale (e.g., unit precision at 1mm), requiring 18-bit geometry precision for storage.

Data Augmentation. In practice, the characteristics of point clouds vary greatly from one to another. Among them, geometry precision is one of the most important factors, because it has a significant impact on the spatial variance, density, and thus the efficiency of compression methods.

It is impractical to assume a fixed geometry precision for raw input, and it is also difficult to set a constant number of scales. Recalling that our solution applies the resolution scaling for multiscale representation, we train SOPA models for two consecutive scales and then apply them for inference across all scales.

In order to generalize the trained models to an excessive amount of dynamic content, we first downscale the point clouds by a random scaling factor in (0.5,1) and then perform dyadic voxel downscaling upon them with the scaling factors at $\{1, \frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \cdots\}$ to obtain ground truth labels at

multiple scales. These ground truth labels are then used to supervise the training of SOPA and SLNE models.

Remark. We specifically use ShapeNet trained model to compress dense object point clouds (e.g., 8iVFB, Owlli), and apply KITTI tuned model for sparse LiDAR point clouds like Ford sequences, when quantitatively measuring the compression efficiency against the G-PCC anchor. This is because very diverse point density and bit precision properties are presented for dense object and sparse LiDAR content, making it inefficient to use a single model for compression. Note that the test sequences exhibit very different content distributions from that of the training set, evidencing the robust generalization of the proposed SparsePCGC (see Table 4).

On the other hand, in order to ensure the fair comparison with other learning-based approaches like VoxelDNN [29], NNOC [27], VCN [13], and OctAttention [28], we have fine-tuned our model using their training sets for performance evaluation and enforced the test using a comparative low-end hardware platform (e.g., RTX 2080) for complexity measurement. We have also faithfully reproduced the best results of these baselines by either executing publicly-accessible source codes [27], [28], or consulting authors for the latest data [13], or directly citing numbers from published paper [10], [29].

5.2 Loss Functions

Basically, the SOPA model estimates the probability of occupancy status $p_{\text{MP-POV}}$ for a given MP-POV at a specific scale, which is then used to code the binary occupancy symbol of this MP-POV in lossless mode, or to perform the binary classification in lossy compression mode.

In general, the Binary Cross-Entropy (BCE) between estimated occupancy probability and actual occupancy symbol is used in training, i.e.,

$$L_{\text{BCE}} = \sum_j -(o(j)\log_2(p(j)) + (1-o(j))\log_2(1-p(j))), \tag{6}$$

where $o(j)$ represents the actual occupancy symbol (e.g., 1 for POV and 0 for NOV), $p = p_{\text{MP-POV}}$ is the probability that the jth MP-POV is POV, and thus (1-p) is the probability of being the NOV.

Note that the SOPA model can be improved by including encoder-side SLNE model. Thus, the loss function for SLNE enhanced SOPA model is formulated as the combination of BCE loss in (6) and rate consumption of feature attributes:

$$L_{\text{comb}} = L_{\text{BCE}} + R_F, \tag{7}$$

with R_F defined in (4).

For sparse LiDAR point clouds using SOPA (Position) model, we treat it slightly different. Given that the SOPA (Position) model directly outputs the occupancy position offset, we measure the Mean Square Error (MSE) between the coordinates of adjusted points and true points in upscaled sparse tensor, i.e.,

$$L_{\text{MSE}} = \frac{1}{J}\sum_j \left((x^N, y^N, z^N)(j) - (\hat{x}^N, \hat{y}^N, \hat{z}^N)(j)\right)^2, \tag{8}$$

having J points in total.

Fig. 10. *Point Cloud Examples.* (a) Dense object point clouds from 8iVFB; (b) Sparse LiDAR point clouds from KITTI.

5.3 Training Strategies

Overall, the training of SOPA models is very straightforward and easy, while jointly optimizing combined loss functions in SLNE enhanced One-Stage SOPA model is slightly difficult. Therefore we propose to set the weight of R_F to 0 in (7) at first, and then gradually increase it to 1 for robust and fast convergence. For SLNE enhanced Multi-Stage SOPA, only the SOPA models are updated, while the SLNE model is directly borrowed from the SLNE enhanced One-Stage model and its parameters are fixed in training.

The learning rate decreases from 0.0008 to 0.00002 in the training. The training iteration executes around 30 epochs with a batch size of 8. Adam is used as the optimizer.

6 Experimental Studies

6.1 Test Setup

Datasets. The MPEG PCC datasets [48] that are recommended and used by standardization committee are tested for performance evaluation as illustrated in Fig. 10.

Dense Point Clouds. We choose several static samples from the Category 1 of MPEG PCC dataset, including longdress_vox10_1300, redandblack_vox10_1550, soldier_vox10_0690, loot_vox10_1200, queen_0200, basketball_player_vox11_0200, and dancer_vox11_0001. They are densely sampled but have different geometry precision and characteristics to represent diverse objects. Later, both 8iVFB and MVUB sequences are used when performing comparative studies with other learning-based approaches.

Sparse Point Clouds. We use KITTI and Ford samples as typical sparse LiDAR point clouds. For KITTI content, we use 0-11 sequences for training (see Sec. 5.1) and 12-21 sequences for testing[5]. The raw floating-point coordinates are quantized to 1mm (18 bits required) and 2cm (14 bits required) precision.

The Ford dataset is used to test dynamically-acquired point clouds as the Category 3 samples in MPEG PCC datasets. It contains 3 sequences (Ford_01, Ford_02, and Ford_03), and each sequence has 1500 frames and averaged 100k points per frame. We use all 3 sequences for testing. The original Ford dataset is already quantized to 1mm (18 bits) precision. Here, we further quantize it to 2cm (14 bits required) precision for an additional test.

More details regarding the MPEG PCC datasets can be found in [48].

5. Such split of training and testing sequences was commonly used in [11], [13], [28].

Test Conditions. For a fair comparison, we exactly follow the MPEG common test conditions (CTC) [49] to generate the anchor results using MPEG G-PCC. The latest reference software TMC13-v14 of G-PCC [20] is used, and the G-PCC octree codec is enabled in study. The augular coding mode is disabled when compressing the LiDAR point clouds based on the assumption that we do not utilize any prior knowledge from point cloud acquisition stage. For the lossless mode of G-PCC, the "positionQuantizationScale" is set to 1, while for the lossy compression mode, it is set to 1/2, 1/4, 1/8, etc., to offer variable bit rates. When evaluating the distortion for lossy compression, both *point-to-point error* (D1) and *point-to-plane error* (D2) are used to derive the PSNR. For lossless compression, the *bits per point* (bpp) is used to measure the compression ratio.

Our model prototype is implemented using PyTorch and MinkowskiEngine [41], which is tested on a computer with an Intel Xeon Silver 4210 CPU and an Nvidia GeForce RTX 2080 GPU. We record the encoding and decoding time following the methodology used in G-PCC. Because of the implementation variation, e.g., CPU vs. GPU, C/C++ vs. Python, the runtime comparison only serves as the intuitive reference to have a general idea about the computational complexity.

6.2 SparsePCGC Presets

The SparsePCGC offers a variety of options to compress the input PCG by applying different settings of SOPA and SLNE models. This section exemplifies the typical configuration for subsequent performance evaluation and comparison.

6.2.1 Lossless Mode

For either dense or sparse point clouds, the SparsePCGC applies the lossless mode for all scales to mandate the perfect reconstruction. For simplicity, we exemplify the trade-off between the computational complexity and compression efficiency by encoding/decoding dense object point clouds in Table 2. Note that computational complexity is presented using the encoding/decoding time averaged per frame.

Choice of SOPA Model. When the SLNE function is not enabled in the encoder, the coding gain over the G-PCC anchor increases greatly from the toy baseline using One-Stage SOPA with 5.5% loss, to the case using 8-Stage SOPA model with 38.5% compression gain. As also detailed in the supplemental material, available online, n-Stage SOPA provides slightly better gain, e.g., 39.3% through the use of autoregressive neighbors in a sequential order. However, the decoding time of n-Stage SOPA is unbearable, e.g., $\approx$2 hrs, due to the causal dependency in computation from one voxel to another.

As seen, the proposed 3-Stage or 8-Stage SOPA model not only provides the encouraging compression efficiency, but also just requires 1$\sim$2 seconds for encoding or decoding a PCG frame averaged among all 8iVFB test frames in Table 2. Quantitatively, our SparsePCGC is faster than the G-PCC anchor, e.g., 1.86s vs. 5.88s of encoding and 1.78s vs. 3.34s of decoding. But these numbers are just used as a reference to indicate that the proposed SparsePCGC is a solution with relatively low computational complexity requirement. This is because C++/C implemented G-PCC mainly runs on CPU while our method prototyped using PyTorch framework runs on GPUs (e.g., RTX 2080 used in comparison).

TABLE 3
Impact of k and C Used in Sparse Convolution for Lossless Compression Examples

kernel size channels	G-PCC	k3 C16	k3 C32	k5 C16	k5 C32
Model Size (MB)	-	1.31	4.89	5.53	21.75
Dense PCGs (8iVFB_vox10)					
bpp	1.029	0.642	**0.633**	0.642	0.655
gain	-	-37.6%	**-38.5%**	-37.6%	-36.3%
EncTime (s)	5.88	**1.41**	1.86	2.45	3.44
DecTime (s)	3.34	**1.28**	1.78	2.45	3.31
Sparse PCGs (KITTI_vox2cm)					
bpp	7.637	7.021	6.880	6.334	**6.130**
gain	**-**	-8.1%	-9.9%	-17.1%	**-19.7%**
EncTime (s)	1.16	**1.15**	1.23	1.40	1.73
DecTime (s)	0.72	1.04	1.12	1.32	1.61

We then explore the potential to use SLNE in the encoder to aggregate local spatial features for better probability estimation in SOPA model. Having the SLNE combined with One-Stage SOPA, the coding efficiency is boosted from 5.5% loss to 25.1% gain, providing evidence to the effectiveness of the SLNE module. The coding gain can be further improved to 35.3% when we combine the SLNE with 3-Stage SOPA, but the relative improvement from 3-Stage SOPA only is limited (e.g., from 31.7% to 35.3%). This relative improvement is negligible when comparing the SLNE with 8-Stage SOPA to the 8-Stage SOPA only. This occurs because the SLNE and multi-stage SOPA apply the similar mechanism to characterize and exploit local neighborhood correlations. Similarly, either "SLNE enhanced One-Stage SOPA" or "SLNE enhanced 3-Stage SOPA" shows low computational complexity by requiring less than 2 seconds to encode or decode a PCG frame on average.

As seen, 8-Stage SOPA and SLNE enhanced 3-Stage SOPA are promising candidates for lossless SparsePCGC because of their low complexity and superior compression performance. But training SOPA model only is much faster than training SLNE and SOPA jointly (about 2$\times$ speedup) from our extensive simulations. To this end, "8-Stage SOPA" is used as the lossless mode of the proposed SparsePCGC.

Previous discussions assume the settings of $k = 3$ and $C = 32$ in SOPA model for the lossless compression of dense 8iVFB point clouds. Next, we report the results when applying different k and C to compress dense and sparse PCGs respectively. Note that we still enforce the lossless coding mode for simplicity.

Setting k & C. We train four 8-Stage SOPA models by setting different k and C to evaluate the compression efficiency on both dense and sparse point clouds. In Table 3, as for dense point clouds, the best result is reported when $k = 3$ and $C = 32$. Using the same $k = 3$, $C = 16$ is also a decent choice with less than 1 absolute percent point drop but results in greater than 70% of model size reduction (4.89 MB vs. 1.31 MB). However, performance drops are observed when using a larger convolutional kernel, e.g., $k = 5$. Such performance drop is not presented on the training dataset (ShapeNet), implying potential model overfitting with more parameters.

TABLE 4
Quantitative Performance Gains to the G-PCC Anchor

Point Clouds	lossless					lossy			
	G-PCC		Ours		CR Gain	G-PCC	Ours	BD-Rate Gain	
	bpp	Time (s) Enc/Dec	bpp	Time (s) Enc/Dec		Time (s) Enc/Dec	Time (s) Enc/Dec	D1	D2
Dense Object									
longdress_vox10	1.015	5.73/3.22	0.625	1.80/1.70	-38.4%	4.84/2.50	0.68/1.19	-94.5%	-89.7%
loot_vox10	0.970	5.36/3.01	0.596	1.75/1.60	-38.6%	4.56/2.64	0.64/1.10	-95.0%	-90.3%
red&black_vox10	1.100	6.72/3.56	0.690	1.68/1.84	-37.3%	4.07/1.89	0.64/1.08	-93.6%	-88%
soldier_vox10	1.030	5.07/3.04	0.628	2.22/1.99	-39.0%	5.65/3.32	0.78/1.40	-94.3%	-89.2%
queen_vox10	0.773	7.34/4.09	0.547	1.92/1.85	-29.2%	4.97/2.51	0.66/1.19	-95%	-90.1%
player_vox11	0.898	23.66/14.95	0.520	6.43/6.07	-42.1%	16.05/5.99	1.12/2.49	-97.2%	-94.9%
dancer_vox11	0.880	19.45/12.63	0.514	5.99/5.67	-41.6%	14.74/6.82	1.17/2.32	-96.9%	-93.9%
Average	0.952	10.48/6.36	**0.589**	**3.11/2.96**	**-38.2%**	7.84/3.81	**0.81/1.54**	**-95.2%**	**-90.9%**
Sparse LiDAR									
KITTI_q2cm	7.637	1.16/0.72	6.130	1.73/1.61	-19.7%	0.85/0.49	1.26/0.89	-33.4%	-44.2%
Ford_q2mm	10.032	0.96/0.59	8.784	1.59/1.47	-12.4%	0.75/0.55	1.31/0.99	-29.8%	-37.6%
KITTI_q1mm	20.154	1.77/1.03	17.971	2.91/1.71	-10.8%	1.26/0.72	1.70/1.28	-29.1%	-36.0%
Ford_q1mm	22.298	1.24/0.74	21.168	2.65/2.47	-5.1%	0.98/0.78	1.72/1.31	-22.0%	-26.8%
Average	15.030	**1.28/0.77**	**13.513**	**2.22/2.06**	**-12.0%**	**0.96/0.64**	1.50/1.12	**-28.6%**	**-36.2%**

BD-Rate measurement is used for lossy mode and compression ratio (CR) gain measured by bits per point (bpp) is used in lossless mode.

For sparse LiDAR point clouds, compression performance improves as k and C increase. About 10 absolute percentage points are reported when comparing the cases with $k = 5$ and the cases with $k = 3$ accordingly. We believe this is because the sparse nature of LiDAR points requires a larger convolutional kernel to capture sufficient neighbors for effective information aggregation and embedding.

In a summary, the proposed SparsePCGC chooses to set k = 3 and C = 32 for dense point clouds, and k = 5 and C = 32 for sparse point clouds. In real-life applications, a preferred setting of k and C can be determined by balancing the complexity and performance tradeoff.

6.2.2 Lossy Mode

As in Fig. 9, in the lossless phase, we directly apply the 8-Stage SOPA model to exploit cross-scale and same-scale multi-stage correlations; while in the lossy phase, we use different methods for dense and sparse point clouds due to their diverse geometry precision and point density.

- For dense PCG, we suggest the SLNE-enhanced One-Stage SOPA to upscale the sparse tensor from the mth to the $(m+1)$th scale and then apply the One-Stage SOPA from $(m+1)$ till the highest scale N;
- For sparse PCG, we apply the SOPA (Position) model to directly upscale the sparse tensor from the mth scale to its highest scale N;

Recalling that m is adapted for various rate-distortion trade-off of lossy SparsePCGC, in our experiments, we set the factor m as {N-1, N-2, N-3} for dense point clouds, and {N-2, N-3, $\cdots$, N-9} for sparse LiDAR point clouds.

6.3 Performance Evaluation and Comparison

6.3.1 Comparison to the G-PCC

We first report performance gains of the proposed SparsePCGC to standardized G-PCC anchor in Table 4.

Compression Ratio (CR) Gain in Lossless Mode. For dense point clouds, our SparsePCGC improves the G-PCC by 38.2% on average and up to 42.1% for basketball_player_vox11_0200. For sparse point clouds, our method still outperforms the G-PCC, although the gains are not as high as that of dense point clouds. This is because sparse point clouds exhibit much sparser spatial distribution and it is relatively difficult to capture and characterize inter-point correlations.

BD-Rate Improvement in Lossy Mode. We also evaluate the lossy compression efficiency of our SparsePCGC to the anchor G-PCC. On average, the proposed SparsePCGC shows 95.2% and 90.9% BD-Rate improvement for dense PCGs and 28.6% and 36.2% gains for sparse PCGs, when the distortion is respectively measured by D1 and D2 metrics. Quantitative gains to the G-PCC anchor are also visualized in Fig. 11 using rate-distortion curves. Qualitative comparisons with G-PCC are presented in Fig. 12, using point clouds colored by reconstruction error. All of these clearly demonstrate the superior efficiency of the proposed SparsePCGC.

Runtime Evaluation. We also measure the encoding and decoding time for the G-PCC anchor and our method in Table 4. Quantitatively, our encoding/decoding is about 2$\sim$ 3$\times$ faster than G-PCC on dense object point clouds, while about 2$\sim$ 3$\times$ slower than G-PCC on sparse LiDAR point clouds in lossless mode. A similar runtime trend is observed in lossy mode. The G-PCC is faster on sparse LiDAR PCGs probably because of its dedicated tools engineered to accelerate the process of isolated points [6], while our method currently treats all points equally in the same framework. We particularly notice that our method reports more than 9$\times$ speedup to that of the G-PCC encoder when encoding dense PCGs in the lossy mode. This is because simple dyadic downscaling and fast One-Stage SOPA are used in the proposed SparsePCGC (see Fig. 9b). Overall, our SparsePCGC is a low-complexity solution, requiring only a few seconds to encode/decode large-scale point clouds with state-of-the-art performance.

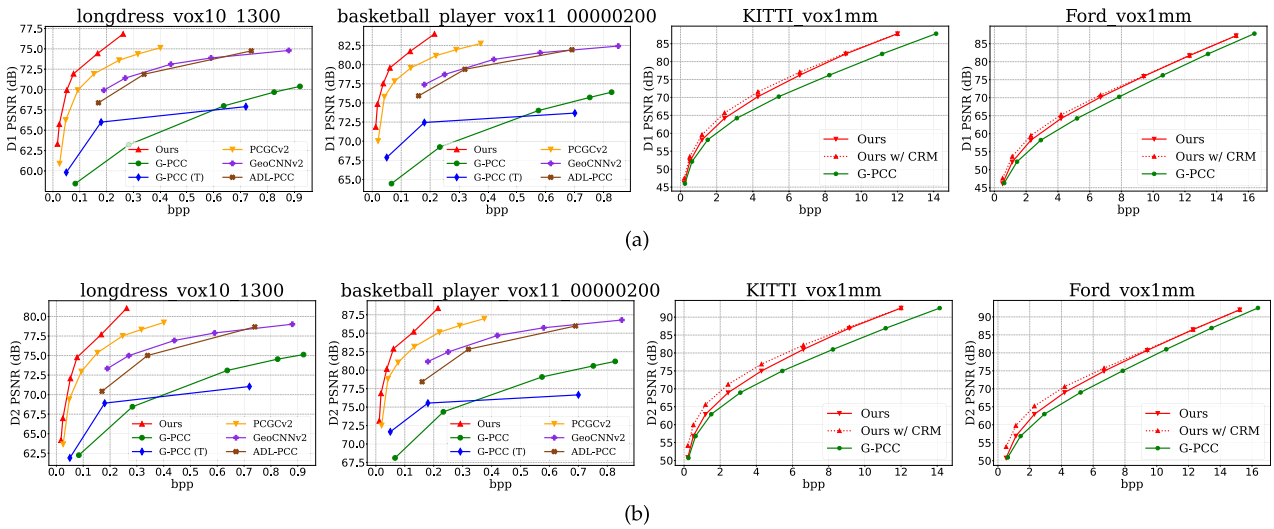


Fig. 11. Performance comparison using rate-distortion curves. (a) D1 PSNR; (b) D2 PSNR. G-PCC anchor using octree codec, G-PCC (T) using trisoup codec, PCGCv2 [22], GeoCNNv2 [26], and ADL-PCC [9] are all compared.

For G-PCC, we collect the runtime from the log file recorded in the reference software. For our method, we include the time for both network inference and entropy engine for bitstream handling. The G-PCC is implemented using C++/C and tested on CPU while our method is written in Python and tested on RTX 2080 GPU. It requires substantial efforts to take implementation details such as CPU vs. GPU, C++/C versus Python, and code optimization levels into the consideration, which is deferred as our future study. Thus, in this work, the runtime measurement and comparison is just an intuitive reference to present a general idea of the computational complexity.

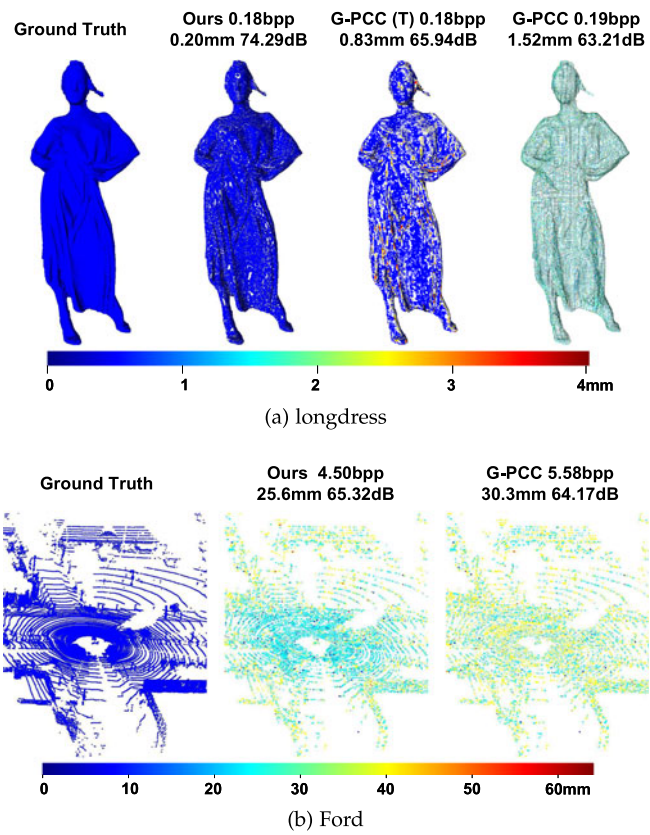


Fig. 12. Qualitative visualization of reconstruction of "longdress_vox10_1300" and "Ford_01_vox1mm_0500" for ground truth, Ours, and G-PCC anchor. The color error map describes the point-to-point distortion measured in mm, and the numbers above represent the bitrate, mean error measured in mm, and D1 PSNR.

6.3.2 *Comparison to other Learning-Based Solutions*

Most learning-based approaches can only deal with a single type of point clouds. Thus, we detail the comparison individually. Again, we reiterate that the encoding/decoding inference runs on RTX 2080 GPU to best match the computation device used by other learning-based solutions. In terms of the model training, we also try our best to apply the common sets used by others for fair comparisons. More details are offered in our supplemental material, available online.

Lossless Compression of Dense PCGs. Nguyen et al. [29] developed the VoxelDNN - a learning-based lossless compression method for dense PCG, in which masked 3D CNN is applied for voxel occupancy probability approximation assuming the use of a uniform voxel representation model. The VoxelDNN provided promising compression ratios at the expense of a very long encoding and decoding time due to the sequential processing. A multiscale parallel version of VoxelDNN, referred to as the MsVoxelDNN [10], was then proposed to speed up the runtime (e.g., 12× for decoding, and 16× for encoding). We directly quote numbers from their papers. It is noted that only one frame is tested for VoxelDNN-related methods.

In contrast to the uniform voxel representation, Kaya et al. [27] proposed an octree-based lossless compression method — NNOC, in which it estimated the occupancy probability of the octree node based on available nodes surrounding it. Similarly, the NNOC also suffered from an extremely long decoding time. Then, a fast version of NNOC, a.k.a., fNNOC, was updated for decoding acceleration (e.g., about 10×). We best reproduce the results using sources from NNOC/fNNOC for evaluation.

Another learning-based octree codec, a.k.a, OctAttention [28] that reported state-of-the-art efficiency through the use of Transformer to effectively characterize and embed

TABLE 5
Performance Comparison of Losslessly Coded Dense Point Clouds

Dense PCs	G-PCC (seqs/one)	Ours	VoxelDNN (one) [10], [29]		NNOC [27]		OctAttention [28]
			VoxelDNN	MsVoxelDNN	NNOC	fNNOC	
8iVFB							
Red&black×300	1.088/1.083	0.635	0.665	0.87	0.723	0.866	0.73
Loot×300	0.961/0.947	0.533	0.577	0.73	0.590	0.743	0.62
Thaidancer×1	0.986	0.557	0.677	0.85	0.684	0.807	0.65
Boxer×1	0.943	0.493	0.550	0.70	0.551	0.682	0.59
Average	0.994/0.990	0.555	0.617	0.788	0.637	0.775	0.648
Gain	-	**-44.2%**	-37.7%	-20.5%	-35.9%	-22.1%	-34.9%
EncTime (s)	5.56/5.49	2.09	(885)	(54)	91	64	**1.06**
DecTime (s)	3.26/3.21	**1.91**	(640)	(58)	1079	66	1229
MVUB							
Phil×245	1.135/1.142	0.714	0.76	1.02	0.782	1.021	0.79
Ricardo×216	1.061/1.103	0.647	0.687	0.95	0.701	0.941	0.72
Average	1.098/1.123	0.681	0.724	0.985	0.742	0.981	0.755
Gain	-	**-37.7%**	-35.5%	-12.5%	-32.1%	-10.2%	-30.9%
EncTime (s)	7.56/9.78	2.81	(885)	(85)	101	66	**1.39**
DecTime (s)	4.53/5.47	**2.64**	(640)	(92)	1318	69	1520

Compression ratio is measured by the bpp gain over G-PCC.

correlations across a large amount of previously-processed neighbors (e.g., parent nodes, sibling nodes) for context modeling was included for comparison as well. The OctAttention achieved fast encoding with just 1∼2 seconds because full data availability in the encoder allowed the use of massive encoding parallelism; however, its decoding was super slow due to causal data dependency. We reproduce the results of the OctAttention model using its publicly-accessible sources.

Table 5 quantitatively compares the lossless compression efficiency of the proposed SparsePCGC, VoxelDNN (MsVoxelDNN), NNOC (fNNOC), and OctAttention. We test sequences in both 8iVFB and MVUB. Except for VoxelDNN solutions that encode/decode one frame for each test sequence (a.k.a, "one"), all other solutions process all frames (a.k.a., "seqs") in each sequence for averaged results[6].

On top of the same G-PCC anchor, our method attains the best gains, e.g., on average, 44.2% for 8iVFB and 37.7% for MVUB sequences. More than 20 absolute percentage points are captured over the MsVoxelDNN and fNNOC. As compared with the OctAttention that requires 1024 neighboring nodes for context modeling, our SparsePCGC that solely relies on local neighbors within a limited receptive field (e.g., $3 \times 3 \times 3$ for dense PCG) provides another 7∼10 absolute percentage points improvement approximately.

As for the encoding/decoding runtime comparison with other learning-based solutions, our method speeds up the MsVoxelDNN/fNNOC at least 20×. Though the OctAttention model provides the fastest encoding, it is still at the same order of magnitude as our method, not to mention that its decoding time is more than 500× slower.

Lossy Compression of Dense PCGs. We then compare the SparsePCGC with the PCGCv2 [22], GeoCNNv2 [26], and ADL-PCC [9] for lossy compression of dense PCGs. Both GeoCNNv2 [26] and ADL-PCC [9] leverage the dense CNNs assuming the use of uniform voxel representation. The PCGCv2 [22] is our earlier attempt to apply the multi-scale sparse tensor, which applies the fixed factors for resolution scaling, uses the lossless G-PCC to code geometry coordinates at the lowest scale, and supports the lossy compression of dense PCGs only. Both rate-distortion curves in Fig. 11 and BD-Rate gains in Table 6 evidence that the SparsePCGC presents significant performance lead over these methods. Note that lossy PCGC approaches are studied extensively in MPEG for the development of next-generation learning-based PCGC standard. A companion document [50] also reports the superior efficiency of the proposed method from the third-party evaluations.

Lossy Compression of Sparse LiDAR PCGs. Most approaches adopt the octree model to represent sparse LiDAR point clouds. Huang et al. [11] proposed the OctSqueeze that used MLP to exploit the dependency between parent and child nodes. Que et al. [13] then proposed the VCN (VoxelContextNet) which arranged sibling nodes using the uniform voxel model and employed 3D CNN to exploit the dependency across nodes. As aforementioned, a CRM was utilized in VCN to improve the reconstruction quality. In addition, OctAttention [28] that presented state-of-the-art efficiency for the compression of lossy sparse LiDAR PCG is also compared.

Since both OctSqueeze [13] and VCN [11] do not offer the sources to the public, it is difficult for us to replicate their results. We directly quote the results from VCN paper (i.e., Fig. 6 in [13]) and present them in Table 7 and Fig. 13. As for the OctAttention, we reproduce the results by running its source code. We best ensure the same testing/training conditions as these methods like VCN and OctAttention for comparative study where the details are specified in the supplemental material, available online.

As seen, our method still achieves leading compression performance when compared with both VCN and OctAttention. For VCN, we also provide its results with the CRM in

6. Only one frame is contained in both "Thaidancer×1" and "Boxer×1" sequences, implying the same results for "one" and "seqs" categories.

TABLE 6
BD-Rate Gains Measured Using Both D1 and D2 for the SparsePCGC Against the G-PCC (T) Using Trisoup Codec Option, PCGCv2 [22], GeoCNNv2 [26], and ADL-PCC [9] for Lossy Coded Dense Point Clouds

Dense PCs	G-PCC (Trisoup)		PCGCv2 [22]		ADL-PCC [9]		GeoCNNv2 [26]	
	D1	D2	D1	D2	D1	D2	D1	D2
longdress_vox10_1300	-76.4%	-74.3%	-47.2%	-36.1%	-76.6%	-74.7%	-74.9%	-70.1%
loot_vox10_1200	-69.6%	-68.0%	-45.8%	-34.8%	-76.4%	-74.3%	-74.9%	-69.6%
red&black_vox10_1550	-72.7%	-70.6%	-45.0%	-28.2%	-69.6%	-68.0%	-72.7%	-66.2%
soldier_vox10_0690	-77.4%	-76.7%	-45.8%	-34.7%	-72.7%	-70.6%	-73.1%	-68.1%
queen_0200	-81.8%	-80.4%	-34.7%	-29.2%	-77.4%	-76.7%	-72.5%	-66.6%
player_vox11_200	-80.6%	-78.2%	-48.5%	-41.9%	-81.8%	-80.4%	-79.5%	-75.7%
dancer_vox11_0001	-76.4%	-74.7%	-49.7%	-42.8%	-80.6%	-78.2%	-78.4%	-73.6%
Average	**-89.2%**	**-81.2%**	**-45.2%**	**-35.4%**	**-76.4%**	**-74.7%**	**-75.1%**	**-70.0%**

TABLE 7
BD-Rate Gains Over the G-PCC Anchor for Lossily Compressed Sparse LiDAR PCG

KITTI	Test Conditions of VCN [13]				Test Conditions of OctAttention [28]		
	G-PCC v8	Ours	VCN	OctSqueeze	G-PCC v14	Ours	OctAttention
BD-Rate	-	**-25.6%**	-16.7%	-2.1%	-	**-22.0%**	-19.5%
BD-Rate w/ CRM	-	**-37.7%**	-30.0%	-	-	-	-
EncTime (s)	1.30	1.44	-	**-**	0.92	1.44	**0.44**
DecTime (s)	0.55	1.32	0.09	**0.008**	**0.62**	1.19	530

Runtime at the highest bitrate point (12-bit) are exemplified for comparison, all methods are tested on RTX 2080 GPU.

comparison to our solution with position offset adjustment. Here we use the same terminology CRM in Table 7 to represent the post-processing module. Clearly, our BD-Rate gains are still retained with CRM.

Moreover, we show the runtime for all methods when using them to compress LiDAR PCG in Table 7. Currently, VCN shows the fastest decoding speed. Our method runs about 1∼2 seconds for both encoding and decoding, presenting about 2× slower than the G-PCC version 14. Because of the causal dependency used in context modeling, OctAttention consumes the longest time for decoding.

6.4 Discussion

Due to various acquisition techniques used in different applications, the characteristics of input point clouds, including the density, volume, precision, noise, etc., vary greatly from one to another. It is extremely challenging to efficiently compress these diverse sources under the same model. As a compromise, existing solutions apply different models or tools to compress different inputs as aforementioned.

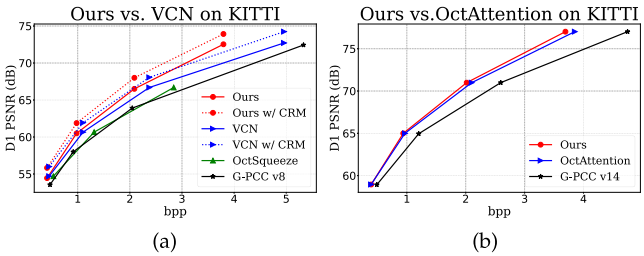


Fig. 13. Performance comparison using rate-distortion curves (a) Ours versus the VCN under the same test conditions used by VCN [13]; (b) Ours versus the OctAttention under the same test conditions used in OctAttention [28].

Previous sections reveal that the SparsePCGC, as a unified model, can compress diverse point clouds in either lossy or lossless mode efficiently. This is mainly because the proposed method can effectively exploit the sparsity nature of unstructured points:

- The SparsePCGC deals with the points directly without assuming any prior knowledge of the captured scene, which ideally supports arbitrary-distributed points in the 3D space. The sparse convolution used in this work can significantly reduce the complexity and best leverage valid neighbors within the receptive field to analyze and aggregate local variations;
- The multiscale representation leverages the resolution re-scaling for scale-wise construction, with which we can exploit cross-scale correlations to effectively aggregate and embed spatial information from local neighborhood for compression.

The proposed SparsePCGC applies model sharing across scales. Lossless SparsePCGC uses the same 8-Stage SOPA model which only consumes 4.9 MB to process either dense or sparse point clouds. The lossy mode of SparsePCGC comprises a lossless and a lossy phase. For the compression of dense PCGs, we use the same 4.9 MB 8-Stage SOPA model in lossless phase, 5.8 MB SLNE enhanced One-Stage SOPA and 0.85 MB One-Stage SOPA in lossy phase; while for the sparse PCGs, the lossless phase requires the 21.7 MB 8-Stage SOPA model since we use a larger kernel size, e.g., 5×5×5, and the lossy phase uses the 31.6 MB SOPA (Position) model. Recently, a number of model compression methods have been developed to reduce the model size [51], such as the pruning and quantization techniques, which is an interesting topic for future study.

7 Conclusion and Future Work

A unified point cloud geometry compression approach is developed, demonstrating state-of-the-art performance in both lossless and lossy compression applications across a variety of datasets, including the dense point clouds (8iVFB, Owlii, MVUB) and the sparse LiDAR point clouds (KITTI, Ford) when compared with the MPEG G-PCC and other popular learning-based approaches. Moreover, the proposed method attains low computational consumption because of the utilization of sparse convolution, and requires a small amount of storage for model parameters, since the same model is shared across scales.

The advantages of the proposed method come from the Multiscale Sparse Tensor Representation, where we use sparse convolutions to directly deal with unstructured points and efficiently aggregate local neighborhood information. We also leverage the multiscale representation to extensively exploit the cross-scale correlations for better context modeling in compression.

There are numerous interesting topics for exploration in the future, including 1) attribute compression (e.g., RGB colors), 2) motion capturing for dynamic point clouds, and 3) better quality metrics close to human perception for loss optimization.

A supplemental material, available online is provided with more details about the experimental setup and additional comparisons at https://github.com/NJUVISION/SparsePCGC/blob/main/Supplementary_Material.pdf.

Acknowledgments

Our sincere gratitude is directed to the authors of relevant works used in comparative studies for discussing experiments in detail, and providing the latest results for evaluation.

References

[1] S. Schwarz et al., "Emerging MPEG standards for point cloud compression," *IEEE Trans. Emerg. Sel. Topics Circuits Syst.*, vol. 9, no. 1, pp. 133–148, Mar. 2019.

[2] H. X. Nguyen, R. Trestian, D. To, and M. Tatipamula, "Digital twin for 5G and beyond," *IEEE Commun. Mag.*, vol. 59, no. 2, pp. 10–15, Feb. 2021.

[3] D. Marpe, H. Schwarz, and T. Wiegand, "Context-based adaptive binary arithmetic coding in the H.264/AVC video compression standard," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 13, no. 7, pp. 620–636, Jul. 2003.

[4] C. E. Shannon, "A mathematical theory of communication," *Bell Syst. Tech. J.*, vol. 27, no. 3, pp. 379–423, 1948.

[5] D. Meagher, "Geometric modeling using octree encoding," *Comput. Graph. Image Process.*, vol. 19, no. 2, pp. 129–147, 1982.

[6] C. Cao, M. Preda, V. Zakharchenko, E. S. Jang, and T. Zaharia, "Compression of sparse and dense dynamic point clouds—methods and standards," *Proc. IEEE*, vol. 109, no. 9, pp. 1537–1558, Sep. 2021.

[7] M. Quach, G. Valenzise, and F. Dufaux, "Learning convolutional transforms for lossy point cloud geometry compression," in *Proc. IEEE Int. Conf. Image Process.*, 2019, pp. 4320–4324.

[8] J. Wang, H. Zhu, H. Liu, and Z. Ma, "Lossy point cloud geometry compression via end-to-end learning," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 31, no. 12, pp. 4909–4923, Dec. 2021.

[9] A. F. R. Guarda, N. M. M. Rodrigues, and F. Pereira, "Adaptive deep learning-based point cloud geometry coding," *IEEE J. Sel. Topics Signal Process.*, vol. 15, no. 2, pp. 415–430, Feb. 2021.

[10] D. Nguyen, M. Quach, G. Valenzise, and P. Duhamel, "Multiscale deep context modeling for lossless point cloud geometry compression," in *Proc. IEEE Int. Conf. Multimedia Expo Workshops*, 2021, pp. 1–6.

[11] L. Huang, S. Wang, K. Wong, J. Liu, and R. Urtasun, "OctSqueeze: Octree-structured entropy model for lidar compression," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 1310–1320.

[12] S. Biswas, J. Liu, K. Wong, S. Wang, and R. Urtasun, "MuSCLE: Multi sweep compression of lidar using deep entropy models," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 22170–22181.

[13] Z. Que, G. Lu, and D. Xu, "VoxelContext-Net: An octree based framework for point cloud compression," *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 6042–6051.

[14] T. Huang and Y. Liu, "3D point cloud geometry compression on deep learning," in *Proc. 27th ACM Int. Conf. Multimedia*, 2019, pp. 890–898.

[15] L. Gao, T. Fan, J. Wang, Y. Xu, and Z. Ma, "Point cloud geometry compression via neural graph sampling," in *Proc. IEEE Int. Conf. Image Process.*, 2021, pp. 3373–3377.

[16] C. Qi, L. Yi, H. Su, and L. Guibas, "PointNet++: Deep hierarchical feature learning on point sets in a metric space," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 5105–5114.

[17] S. Perry, "JPEG pleno point cloud coding common test conditions v3.6," ISO/IEC JTC1/SC29/WG11 (JPEG) output document N91058, Apr. 2019.

[18] J. Pang, M. A. Lodhi, G. Martin-Cocher, and D. Tian, "AI-based point cloud compression for new PCC," ISO/IEC JTC1/SC29/WG11 (MPEG) input document m56776, Apr. 2021.

[19] B. Graham, "Sparse 3D convolutional neural networks," in *Proc. Brit. Mach. Vis. Conf.*, 2015, pp. 50.1–150.9.

[20] MPEG-PCC-TMC13, 2021. [Online]. Available: https://github.com/MPEGGroup/mpeg-pcc-tmc13

[21] G. Bjøntegaard, "Calculation of average PSNR differences between rd-curves," in *Proc. 13th VCEG Meeting*, 2001.

[22] J. Wang, D. Ding, Z. Li, and Z. Ma, "Multiscale point cloud geometry compression," in *Proc. Data Compression Conf.*, 2021, pp. 73–82.

[23] E. D'Eon, B. Harrison, T. Myers, and P. A. Chou, "8I voxelized full bodies - a voxelized point cloud dataset," ISO/IEC JTC1/SC29 Joint WG11/WG1 (MPEG/JPEG) input document m40059/M74006, Jan. 2017.

[24] Y. Xu, Y. Lu, and Z. Wen, "Owlii dynamic human mesh sequence dataset," ISO/IEC JTC1/SC29/WG11 (MPEG) input document m41658, Oct. 2017.

[25] C. Loop, Q. Cai, S. O. Escolano, and P. A. Chou, "Microsoft voxelized upper bodies - a voxelized point cloud dataset," ISO/IEC JTC1/SC29 Joint WG11/WG1 (MPEG/JPEG) input document m38673/M72012, May 2016.

[26] M. Quach, G. Valenzise, and F. Dufaux, "Improved deep point cloud geometry compression," in *Proc. 22nd Int. Workshop Multimedia Signal Process.*, 2020, pp. 1–6.

[27] E. C. Kaya and I. Tabus, "Neural network modeling of probabilities for coding the octree representation of point clouds," in *Proc. IEEE 23rd Int. Workshop Multimedia Signal Process.*, 2021, pp. 1–6.

[28] C. Fu, G. Li, R. Song, W. Gao, and S. Liu, "OctAttention: Octree-based large-scale contexts model for point cloud compression," in *Proc. AAAI Conf. Artif. Intell.*, 2022, pp. 625–633.

[29] D. Nguyen, M. Quach, G. Valenzise, and P. Duhamel, "Lossless coding of point cloud geometry using a deep generative model," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 31, no. 12, pp. 4617–4629, Dec. 2021.

[30] C. L. Jackins and S. L. Tanimoto, "Oct-trees and their use in representing three-dimensional objects," *Comput. Graph. Image Process.*, vol. 14, no. 3, pp. 249–270, 1980.

[31] R. Schnabel and R. Klein, "Octree-based point-cloud compression," in *Proc. 3rd Eurograph./IEEE VGTC Conf. Point-Based Graph. (SPBG)*, 2006, pp. 111–121.

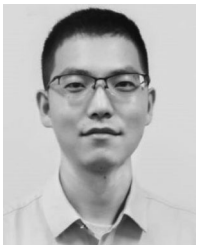
[32] Y. Huang, J. Peng, C. C. Kuo, and M. Gopi, "A generic scheme for progressive point cloud coding," *IEEE Trans. Vis. Comput. Graphics*, vol. 14, no. 2, pp. 440–453, Mar./Apr. 2008.

[33] D. Graziosi, O. Nakagami, S. Kuma, A. Zaghetto, T. Suzuki, and A. Tabatabai, "An overview of ongoing point cloud compression standardization activities: Video-based (V-PCC) and geometry-based (G-PCC)," *APSIPA Trans. Signal Inf. Process.*, vol. 9, no. 1, 2020.

[34] B. Bross et al., "Overview of the versatile video coding (VVC) standard and its applications," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 31, no. 10, pp. 3736–3764, Oct. 2021.

[35] W. Zhu, Z. Ma, Y. Xu, L. Li, and Z. Li, "View-dependent dynamic point cloud compression," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 31, no. 2, pp. 765–781, Feb. 2020.

[36] C. Choy, J. Gwak, and S. Savarese, "4D spatio-temporal convnets: Minkowski convolutional neural networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 3075–3084.
[37] M. Ren, A. Pokrovsky, B. Yang, and R. Urtasun, "SBNet: Sparse blocks network for fast inference," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 8711–8720.
[38] Y. Yan, Y. Mao, and B. Li, "SECOND: Sparsely embedded convolutional detection," *Sensors*, vol. 18, 2018, Art. no. 3337.
[39] B. Graham and L. van der Maaten, "Submanifold sparse convolutional networks," 2017, *arXiv:1706.01307*.
[40] H. Tang, Z. Liu, X. Li, Y. Lin, and S. Han, "TorchSparse: Efficient point cloud inference engine," in *Proc. Conf. Mach. Learn. Syst.*, 2022, pp. 302–315.
[41] Minkowskiengine, 2021. https://github.com/NVIDIA/MinkowskiEngine
[42] C. Szegedy, S. Ioffe, V. Vanhoucke, and A. A. Alemi, "Inception-v4, inception-resnet and the impact of residual connections on learning," in *Proc. AAAI Conf. Artif. Intell.*, 2017, pp. 4278–4284.
[43] D. Minnen, J. Ballé, and G. D. Toderici, "Joint autoregressive and hierarchical priors for learned image compression," in *Proc. IEEE Proc. Int. Conf. Neural Inf. Process. Syst.*, 2018, pp. 10771–10780.
[44] T. Chen, H. Liu, Z. Ma, Q. Shen, X. Cao, and Y. Wang, "End-to-end learnt image compression via nonlocal attention optimization and improved contenxt modeling," *IEEE Trans. Image Process.*, vol. 30, pp. 3179–3191, 2021, doi: 10.1109/TIP.2021.3058615.
[45] J. Ballé et al., "Variational image compression with a scale hyperprior," in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 1–23.
[46] A. X. Chang et al., "ShapeNet: An information-rich 3D model repository," 2015, *arXiv:1512.03012*.
[47] J. Behley et al., "SemanticKITTI: A dataset for semantic scene understanding of lidar sequences," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2019, pp. 9296–9306.
[48] MPEG PCC dataset, 2021. http://mpegfs.int-evry.fr/mpegcontent/
[49] "Common test conditions for G-PCC," ISO/IEC JTC1/SC29/WG7 (MPEG 3DG) output document N00106, Apr. 2021.
[50] A. Zaghetto et al., "Performance analysis of currently AI-based available solutions for PCC," ISO/IEC JTC1/SC29/WG7 (MPEG 3DG) output ducument N00174, Jul. 2021.
[51] W. Hong, T. Chen, M. Lu, S. Pu, and Z. Ma, "Efficient neural image decoding via fixed-point inference," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 31, no. 9, pp. 3618–3630, Sep. 2020.



Jianqiang Wang received the BS and MS degrees in electronic science and engineering from Nanjing University, Jiangsu, China, in 2018 and 2021. He is currently working toward the PhD degree in Nanjing University. His research interests include point cloud compression and processing.

Dandan Ding received the BEng degree (Honor.) in communication engineering from Zhejiang University, China, in 2006. From 2007 to 2008, she was an exchange student in Microelectronic Systems Laboratory (GR-LSM) of EPFL, Switzerland. After returning to Zhejiang University, she earned the PhD degree, in June 2011, and served first as a postdoc researcher from July 2011 through June 2013, then as a research associate in Zhejiang University till 2015. Since 2016, she has been with the School of Information Science and Technology, Hangzhou Normal University as a faculty member with tenure track. Her research interests include artificial intelligence based image/video processing, video coding algorithm design and optimization, and point cloud processing. Currently, she is active in the development of new video coding algorithms for the new generation video coding standards, such as AOM/AV2.

Zhu Li is (Senior Member, IEEE) received the PhD degree in electrical and computer engineering from Northwestern University, Evanston, in 2004. He is now an associate professor with the Department of computer science and electrical engnieering (CSEE), University of Missouri,Kansas City, and director of the NSF I/UCRC Center for Big Learning (CBL) with UMKC. He was AFOSR SFFP summer visiting faculty at the US Air Force Academy (USAFA), 2016 , 2017 , 2018 and 2020, with the UAV Research Center. He was Sr. Staff Researcher/Sr. Manager with Samsung Research America's Multimedia Standards Research Lab in Richardson, TX, 2012-2015, Sr. Staff Researcher/Media Analytics Lead with FutureWei (Huawei) Technology's Media Lab in Bridgewater, NJ, 2010 2012, and an assistant professor with the Department of Computing, The Hong Kong Polytechnic University from 2008 to 2010, and a Principal Staff Research Engineer with the Multimedia Research Lab (MRL), Motorola Labs, from 2000 to 2008. His research interests include point cloud and light field compression, graph signal processing and deep learning in the next gen visual compression, image processing and understanding. He has 47 issued or pending patents, more than 100 publications in book chapters, journals, and conferences in these areas. He is an Associated Editor for IEEE Trans on Image Processing (2020), IEEE Trans. on Multimedia (2015-18), IEEE Trans on Circuits and System for Video Technology(2016-19), and Journal of Signal Processing Systems (Springer), since 2015. He serves on the steering committee member of IEEE ICME (2015-18), he is an elected member of the IEEE Multimedia Signal Processing (MMSP), IEEE Image, Video, and Multidimensional Signal Processing (IVMSP), and IEEE Visual Signal Processing and Communication (VSPC) Tech Committees. He is program co-chair for IEEE Intl Conf on Multimedia and Expo (ICME) 2019, and co-chaired the IEEE Visual Communication and Image Processing (VCIP) 2017. He received the Best Paper Award with IEEE Int'l Conf on Multimedia and Expo (ICME), Toronto, 2006, the Best Paper Award (DoCoMo Labs Innovative Paper) with IEEE Int'l Conf on Image Processing (ICIP), San Antonio, 2007.

Xiaoxing Feng manages the research institute of Jiangsu Longyuan Zhenhua Marine Engineering Co., LTD. His research interests include the development of marine engineering instruments and ultra low bandwidth video communication.

Chuntong Cao is the General Manager of Jiangsu Longyuan Zhenhua Marine Engineering Co., LTD. He oversees all engineering developments in marine explorations. His current focuses are digitalizing the process of marine engineering for safe and sustainable development, for which technologies including the digital twin, virtual reality, visual communications, etc are intensively involved.

Zhan Ma (Senior Member, IEEE) received the BS and MS degrees from the Huazhong University of Science and Technology, Wuhan, China, in 2004 and 2006 respectively, and the PhD degree from the New York University, New York, in 2011. He is now on the faculty of Electronic Science and Engineering School, Nanjing University, Nanjing, Jiangsu, 210093, China. From 2011 to 2014, he has been with Samsung Research America, Dallas TX, and Futurewei Technologies, Inc., Santa Clara, CA, respectively. His research focuses on the learning-based video coding, and smart cameras. He is a co-recipient of the 2018 PCM Best Paper Finalist, 2019 IEEE Broadcast Technology Society Best Paper Award, and 2020 IEEE MMSP Image Compression Grand Challenge Best Performing Solution.